Define la arquitectura de verificación mediante **4 Agentes de Auditoría Especializados** y **1 Árbitro de Certificación Final**, especificando exactamente **qué debe auditar cada agente, en qué archivos, bajo qué invariantes matemáticos/pedagógicos y con qué comandos ejecutables** para certificar los Bloques 1 a 5 de docs/SANITATION_WORKFLOW_AUDIT.md (v2.0).

Al final de la respuesta encontrarás el **bloque único en .md puro (sin cortes)** para copiarlo en un solo clic o exportarlo a Google Docs/PDF.

Protocolo de Despliegue de Agentes de Auditoría y Certificación Integral (v2.0)

- **Archivo destino sugerido:** docs/MULTI_AGENT_AUDIT_PROTOCOL.md
-
- **Referencia normativa:** docs/SANITATION_WORKFLOW_AUDIT.md (v2.0) y docs/AUDIT_FINDINGS.md
-
- **Criterio de Aprobación Global: Cero regresiones**, **0 referencias a código muerto**, **100% de invariantes de los hallazgos** H-01 **a** H-10 **verificados**, **0 errores de compilación/linting** y **0 errores HTTP 500 en fuzzing**.
-

1. Topología de Despliegue de Agentes Auditores

Para evitar puntos ciegos y falsos positivos, la auditoría no debe ejecutarse como una revisión superficial, sino dividirse en **4 Agentes Auditores Independientes** con responsabilidades ortogonales:

Agente	Dominio de Auditoría	Hallazgos que Certifica	Herramientas / Harness de Verificación
Agente 1 (AUDIT-PURGE-DB**)**	Purga de Código Muerto, Esquema SQLite y Contratos DTO	H-05, H-07, H-09 y Micro-ajustes 1–2	Búsqueda estática (Select-String), inspección de tablas/índices SQLite y validación Pydantic v2 / TypeScript.

Agente 2 (AUDIT-DAG-TX)	Cola Topológica, Transaccionalidad en Anki y Vínculos del DAG	H-01, H-03, H-10 y Micro-ajuste 3	pytest sobre SanitationService, DeckDAGService, ExportDeckUseCase e inyección de fallos en AnkiConnect.
Agente 3 (AUDIT-PEDAGOGY-AI)	Memoria de Colisiones (Ledger), Multimodalidad (1–8 Imágenes) y Reto Socrático	H-02, H-04, H-05, H-08	pytest sobre DeckIntelligenceService, InspectMaterialUseCase, SocraticRedTeamingService y Schemathesis API Fuzzing.
Agente 4 (AUDIT-FE-E2E)	Unificación en Ventana Principal, Seguridad de UI y Flujos E2E	H-05, H-06, H-07, H-10	tsc --noEmit, oxlint, vite build y suite Playwright E2E (flow_a a flow_d).

2. Instrucciones Detalladas por Agente: ¿Qué Auditar y Cómo?

Agente 1: Auditor de Purga de Código Muerto, Esquema SQLite y Contratos (AUDIT-PURGE-DB)

A. ¿Qué debe auditar?

1. **Eliminación física y cero referencias huérfanas (H-07):**
2.
    - Confirmar que frontend/src/components/ClusterSanitationModal.tsx y frontend/src/components/RefactorNoteModal.tsx ya no existen en el disco ni están importados en DeckStudioModal.tsx o App.tsx.

    -
    - Confirmar que los 3 endpoints obsoletos de saneamiento de clúster (`sanitation-preview`, `sanitize`, `generate-candidates`) y los métodos `sanitize_cluster` y `generate_note_sanitation_candidates` fueron eliminados de `backend/app/api/routes.py`, `backend/app/services/sanitation_service.py` y `frontend/src/services/api.ts`.
    -
    - Confirmar que en `frontend/src/App.tsx` no existen referencias a `isSanitationMode`, `ankicraft_sanitation_mode`, `'Oracle Cloud'` hardcodeado, `pendingObsoleteNoteIds` ni `mediaToInherit.shift()`.
    -
3. **Esquema SQLite y Migración de Estados (`H-04`, `H-09`, Decisión #3)**:
4.
    - Verificar en `backend/app/adapters/db/deck_knowledge_store.py` que `_init_db()` cree la tabla `sanitation_coverage_ledger` y su índice `idx_ledger_pending_note`.
    -
    - Verificar que `concept_nodes` posea la columna `stage_id TEXT DEFAULT 'stage_1'` y el índice `idx_concept_nodes_stage`.
    -
    - Verificar que la migración convierta cualquier registro `quality_status = 'preevaluated'` en `'optimal'`, dejando únicamente los 3 estados canónicos (`unreviewed`, `bad`, `optimal`).
    -
5. **Paridad de Contratos DTO (Backend Pydantic ↔ Frontend TypeScript)**:
6.
    - `Flashcard`: presencia de `source_images: List[str]`.
    -
    - `CoreTheme`: presencia de `diagram_roles: Dict[str, str]` y `associated_diagrams: List[str]`.
    -
    - `LearningPlanContract`: presencia de `LearnerExpertiseLevel`, `CurriculumStage` y `stages: List[CurriculumStage]`.
    -
    - `MatchStudyPlanRequest`: presencia de `source_note_id: Optional[int]`.
    -
    - `TransactionalReplaceRequest`: presencia de `target_concepts` y `pending_ledger_collisions`.
    -

B. ¿Cómo debe auditarlo? (Comandos de Verificación)

- **Barrido de residuos en Frontend y Backend (debe devolver 0 coincidencias):**
- `Select-String -Path "frontend\src\**\*.ts*","backend\app\**\*.py" -Pattern "ClusterSanitationModal|RefactorNoteModal|ankicraft_sanitation_mode|isSanitati`

onMode|pendingObsoleteNoteIds|mediaToInherit\.shift|sanitize_cluster|generate_note_sanitation_candidates"

-
-
- **Prueba de esquema y CRUD del Ledger en SQLite:**
- pytest backend/tests/test_deck_knowledge_store.py -v
-
-
- *(El agente debe verificar que existan tests explícitos para* record_ledger_coverage*,* get_ledger_entries_for_note*,* delete_ledger_entries_for_note *y la migración de* preevaluated → optimal*)*.
-

Agente 2: Auditor de Topología DAG, Cola de Saneamiento y Vía Única Transaccional (AUDIT-DAG-TX)

A. ¿Qué debe auditar?

1. **Ordenamiento Topológico Determinista sin LLM (H-01)**:
2.
    - Auditar SanitationService.get_sanitation_queue(deck_name) en backend/app/services/sanitation_service.py.
    -
    - Comprobar que construya la clave de ordenamiento en 4 niveles: (1) pertenencia a la etapa activa (stage_id), (2) min_depth_level ASC en el DAG (raíces 0 primero, notas no mapeadas = 999), (3) severidad ('bad' antes que 'unreviewed'), (4) índice en topological_order.
    -
    - Comprobar que GET /anki/decks/{deck_name}/sanitation-queue realice **0 llamadas a modelos LLM**.
    -
3. **Integridad Transaccional y Persistencia de Vínculos en el DAG (H-03)**:
4.
    - Auditar SanitationService.replace_note_transactionally:
    -
        1. Llama a _anki_bridge.create_notes y aborta inmediatamente (lanzando RuntimeError sin llamar jamás a delete_notes) si AnkiConnect devuelve lista vacía o falla.
        2.
        3. Elimina la nota monolítica original mediante _anki_bridge.delete_notes([source_note_id]).
        4.

5. Calcula los vectores reales con `_embedding_port.embed_texts` y guarda las nuevas tarjetas en `indexed_notes` con `quality_status = "optimal"`.
6.
7. Rescata los `concept_id` previos de `source_note_id` (más los `target_concepts` aprobados) e inserta en `card_concept_mappings` las filas `(deck_name, new_note_id, concept_id, 'target')` para cada `created_note_id`.
8.
9. Registra las colisiones diferidas en `sanitation_coverage_ledger` y ejecuta `delete_ledger_entries_for_note(deck_name, source_note_id)` para limpiar las entradas consumidas por la nota que acaba de sanearse.
10.
11. Invoca `get_deck_dag(deck_name, recompute_depths=True)` y verifica que el nodo del concepto **no caiga a** `UNMAPPED`, sino que avance a `SANITIZED` cuando todas sus notas asociadas sean `optimal`.
12.

5. **Vía Única de Reemplazo (**H-10**)**:
6.

    - Auditar `backend/app/services/export_deck_service.py` y `backend/app/api/routes.py` (`POST /cards/export/anki`) para confirmar que ya no aceptan ni procesan `obsolete_note_ids` ni invocan `delete_notes`, e indexan las tarjetas nuevas directamente como `optimal`.
    -

B. ¿Cómo debe auditarlo? (Comandos de Verificación)

- **Ejecución de pruebas de saneamiento, topología e inyección de fallos:**
- `pytest backend/tests/test_sanitation_service.py backend/tests/test_deck_dag_service.py backend/audits/test_I1_topological_dag.py backend/audits/test_I3_sanitizer_safety.py -v`
-
-
- **Invariante obligatorio a comprobar por el Agente 2:** Crear un test o escenario donde un nodo con `depth_level = 0` tiene 1 nota `bad` (#100) y un nodo hijo con `depth_level = 1` tiene 1 nota `bad` (#200). Verificar que `get_sanitation_queue` devuelva primero la `#100`, y que al ejecutar `replace_note_transactionally` sobre `#100` con 3 tarjetas atómicas (`#101, #102, #103`), el nodo raíz en `get_deck_dag` conserve `card_count == 3` y pase a `status == "SANITIZED"`.
-

Agente 3: Auditor de Memoria de Colisiones, Inspección Multimodal y Reto Socrático (AUDIT-PEDAGOGY-AI)

A. ¿Qué debe auditar?

1. **Memoria Contextual Diferida en** match_study_plan **(**H-04 **y** H-06**)**:
2.
    - Auditar DeckIntelligenceService.match_study_plan en backend/app/services/deck_intelligence_service.py:
    -
        - Si existen registros en get_ledger_entries_for_note(deck_name, source_note_id) para un eje temático, el eje debe devolverse automáticamente con status = PrerequisiteMatchStatus.DOMINATED, selected = False, recommended_cards = 0 y explicar en verdict_explanation qué tarjetas del turno previo lo cubrieron.
        -
        - Si un eje colisiona con una nota existente B en estado bad o unreviewed, debe devolverse con status = PrerequisiteMatchStatus.COGNITIVE_DEBT, should_replace = False (para que jamás se borre a B en el turno de A) y mantener el eje seleccionado (selected = True) en el turno de A.
        -
    - Auditar GenerateCardsUseCase._apply_mutual_negative_boundaries (backend/app/services/generate_cards_service.py) y get_cluster_sibling_boundaries (deck_knowledge_store.py) para confirmar que **solo las notas vecinas en estado** optimal actúen como barrera negativa de prohibición.
    -
3. **Herencia Multimodal Inteligente de 1 a 8 Imágenes (**H-02 **y** H-05**)**:
4.
    - Auditar InspectMaterialUseCase.execute (backend/app/services/inspect_material_service.py) y el adaptador de generación para confirmar que:
    -
        - Extrae hasta 8 imágenes con retrieve_media_base64 y pasa sus nombres de archivo reales al modelo.
        -
        - Inyecta el nivel del estudiante (target_learner_level de la etapa activa en LearningPlanContract, por defecto beginner con rigor de primeros principios).
        -
        - Devuelve en cada CoreTheme su lista exacta associated_diagrams y el mapa diagram_roles, y en cada Flashcard generada su lista source_images: List[str].
        -
5. **Certificación Dual y Reto Socrático Interactivo por Nodo (**H-08**)**:

6.

- Auditar backend/app/services/socratic_red_teaming_service.py y los endpoints en backend/app/api/routes.py:
-

  - POST /anki/decks/{deck_name}/nodes/{concept_id}/socratic-challenge: verifica que el nodo esté en estado SUFFICIENT (100% optimal + 0 brechas de principiante) y genera el reto.
  -
  - POST /anki/decks/{deck_name}/nodes/{concept_id}/socratic-evaluate:
  -

    - Si score >= 0.75: actualiza el nodo en concept_nodes a status = "MASTERED".
    -
    - Si score < 0.75 por brecha del mazo: inserta un OrphanGap en el nodo y regresa el estado a SANITIZED.
    -
    - Si score < 0.75 por olvido del usuario (concepto ya cubierto por una tarjeta optimal del nodo): conserva el estado SUFFICIENT y devuelve la tarjeta de referencia con su bloque ⚠️ Trampa frecuente.
    -

B. ¿Cómo debe auditarlo? (Comandos de Verificación)

- **Ejecución de suites de inteligencia de mazo, discriminador pedagógico, juez LLM, socrático y fuzzing:**
- pytest backend/tests/test_deck_intelligence_service.py backend/tests/test_pedagogical_discriminator.py backend/audits/test_l2_cognitive_linters.py backend/audits/test_l4_llm_judge_evals.py backend/audits/test_l5_socratic_red_teaming.py backend/audits/test_api_fuzzing_schemathesis.py -v
-
-

Agente 4: Auditor de Frontend, Ergonomía Cognitiva y Flujos E2E (AUDIT-FE-E2E)

A. ¿Qué debe auditar?

1. **No-Destrucción de Notas Colisionadas e Interfaz Multimodal en** MaterialInspectionCockpit.tsx **(**H-05**,** H-06**)**:
2.

- Verificar en handleApproveInspection (frontend/src/components/MaterialInspectionCockpit.tsx) que obsoleteIds **solo contenga** inspection.source_note_id (si la casilla de retirar nota origen está activa) y que **nunca agregue** p.match_info.matched_note_id **ni** t.match_info.matched_note_id.
-
- Verificar que en cambio construya la lista pending_ledger_collisions con las notas cognitive_debt detectadas para enviarla en la sustitución transaccional.
-
- Verificar que renderice la insignia *"🟢 Cubierto en Turno Previo"* cuando el eje provenga del ledger, y que permita reasignar o alternar las imágenes (1..8) asociadas a cada CoreTheme.
-

3. **Unificación del Flujo y Vía Única de Commit en** App.tsx **(**H-05**,** H-07**,** H-10**)**:
4.

- Verificar que la ventana principal muestre la cola topológica conectada a api.getSanitationQueue(options.deck_name).
-
- Verificar que al aprobar/exportar las tarjetas generadas:
-
    - Si existe nota origen (selectedAnkiCard o inspection?.source_note_id), llame exclusivamente a api.replaceNoteTransactionally.
    -
    - Si no existe nota origen, llame a api.exportToAnki (sin obsolete_note_ids).
    -
- Verificar que las imágenes de card.source_images se inyecten de forma determinista por tarjeta sin usar .shift().
-
- Verificar que cuando replaceNoteTransactionally indique que un nodo del DAG completó su saneamiento y suficiencia, se abra SocraticNodeMasteryModal.tsx.
-

5. **Hand-off desde** DeckStudioModal.tsx **y Selector** AnkiCardPickerModal.tsx:
6.

- Verificar que en DeckStudioModal.tsx el botón de cada eje temático sea *"Sanear este Eje en Cola Principal"*, cerrando el modal y posicionando la cola en App.tsx.
-
- Verificar que seleccionar una tarjeta en AnkiCardPickerModal.tsx la cargue en el flujo de *"Analizar y Planificar"* de la ventana principal.
-

B. ¿Cómo debe auditarlo? (Comandos de Verificación)

1. **Verificación de Tipos TypeScript (Cero errores y cero variables sin usar):**
2. `cd frontend && npx tsc -p tsconfig.app.json --noEmit --noUnusedLocals --noUnusedParameters`
3.
4.
5. **Verificación de Linter y Reglas de Ciclo de Vida React (`oxlint`):**
6. `cd frontend && npm run lint`
7.
8.
9. **Compilación de Producción y Code-Splitting:**
10. `cd frontend && npm run build`
11.
12.
13. **Suite E2E Playwright (Flujos A, B, C y D sin `sessionStorage` sintético en Flujo B):**
14. `cd frontend && npm run test:e2e`
15.
16.

3. Bloque Único en Formato `.md` Puro (Copia Completa en 1 Clic)

Usa el botón **"Copiar código"** arriba a la derecha del siguiente recuadro para pegar este documento directamente en `docs/MULTI_AGENT_AUDIT_PROTOCOL.md` dentro de tu workspace:

```Markdown

> **Archivo**: `docs/MULTI_AGENT_AUDIT_PROTOCOL.md`
> ***Especificación Base***: `docs/SANITATION_WORKFLOW_AUDIT.md` (v2.0)
> ***Objetivo***: Desplegar 4 Agentes de Auditoría Especializados + 1 Árbitro de Certificación para comprobar empíricamente que los Bloques 1 a 5 y los hallazgos `H-01` a `H-10` están 100% implementados, libres de código muerto y pasando todas las suites de pruebas.

---

## Reglas de Operación para los Agentes Auditores

1. ***Prohibido asumir por lectura superficial***: Ningún agente puede dar un punto por aprobado sin ejecutar el comando de verificación o el test automatizado correspondiente.
2. ***Independencia de dominio***: Cada agente audita su capa asignada y emite una matriz de veredicto (`PASS` / `FAIL`) con archivo, número de línea y salida de consola.
3. ***Política de Tolerancia Cero***: Un solo `FAIL` en cualquiera de los 4 agentes bloquea la certificación final y exige corrección inmediata antes de cerrar la auditoría.
```

AGENTE 1: `AUDIT-PURGE-DB` (Purga de Código Muerto, Esquema SQLite y Contratos DTO)

1.1. Qué Auditar
- **Eliminación física de archivos obsoletos (`H-07`)**:
  - Verificar que NO existan `frontend/src/components/ClusterSanitationModal.tsx` ni `frontend/src/components/RefactorNoteModal.tsx`.
- **Purga de endpoints, métodos y estados huérfanos (`H-05`, `H-07`, `H-10`)**:
  - En `backend/app/api/routes.py` y `backend/app/services/sanitation_service.py`: verificar eliminación de `get_cluster_sanitation_preview`, `sanitize_cluster` y `generate_note_sanitation_candidates`.
  - En `frontend/src/App.tsx`: verificar ausencia total de `isSanitationMode`, `ankicraft_sanitation_mode`, `'Oracle Cloud'` hardcodeado en `useEffect`, `pendingObsoleteNoteIds` y `mediaToInherit.shift()`.
  - En `backend/app/services/export_deck_service.py`: verificar que `export_to_anki` ya no reciba ni borre `obsolete_note_ids`.
- **Esquema SQLite y Migraciones (`H-04`, `H-09`, Decisión #3)**:
  - En `backend/app/adapters/db/deck_knowledge_store.py`:
    - Existencia de la tabla `sanitation_coverage_ledger` y su índice `idx_ledger_pending_note`.
    - `record_ledger_coverage` `get_ledger_entries_for_note` `delete_ledger_entries_for_note`
    - `stage_id TEXT DEFAULT 'stage_1'` `concept_nodes`
    - `UPDATE indexed_notes SET quality_status = 'optimal' WHERE quality_status = 'preevaluated'`
-

  - `Flashcard.source_images: List[str]`
  - `CoreTheme.diagram_roles: Dict[str, str]` `associated_diagrams: List[str]`
  - `LearningPlanContract.stages: List[CurriculumStage]` `LearnerExpertiseLevel`
  - `ConceptNode.stage_id: str`
  - `MatchStudyPlanRequest.source_note_id: Optional[int]`
  - `TransactionalReplaceRequest.target_concepts` `pending_ledger_collisions`

1. `Test-Path "frontend/src/components/ClusterSanitationModal.tsx"; Test-Path "frontend/src/components/RefactorNoteModal.tsx"` `False`
2. `Select-String -Path "frontend/src/**/*.ts*","backend/app/**/*.py" -Pattern "ClusterSanitationModal|RefactorNoteModal|ankicraft_sanitation_mode|isSanitationMode|pendingObsoleteNoteIds|mediaToInherit\.shift|sanitize_cluster|generate_note_sanitation_candidates"`
3. `pytest backend/tests/test_deck_knowledge_store.py backend/tests/test_domain_and_adapters.py -v`

-
  - `backend/app/services/sanitation_service.py`

    1. `stage_id`
    2. `min_depth_level ASC` `depth = 0`
`depth = 999`
    3. `'bad'` `'unreviewed'`
    4. `get_topological_card_order(deck_name)`
  - `get_sanitation_queue`
-
  - `_anki_bridge.create_notes`
`RuntimeError` `delete_notes`
  - `_anki_bridge.delete_notes([source_note_id])`
  - `_embedding_port.embed_texts`
`indexed_notes` `quality_status = "optimal"`
  - `concept_id` `target_concepts`
`created_note_ids` `card_concept_mappings` `role = 'target'`
  - `pending_ledger_collisions` `sanitation_coverage_ledger`
`delete_ledger_entries_for_note(deck_name,
source_note_id)`
  - `get_deck_dag(deck_name, recompute_depths=True)`
`SANITIZED` `UNMAPPED`

1.
   `pytest backend/tests/test_sanitation_service.py backend/tests/test_deck_dag_service.py
backend/audits/test_l1_topological_dag.py backend/audits/test_l3_sanitizer_safety.py -v`
2. `replace_note_transactionally`
   - `card_concept_mappings` `created_note_ids`
   - `ConceptNode.card_count > 0` `ConceptNode.status == "SANITIZED"`

-
  - `backend/app/services/deck_intelligence_service.py`
    - `get_ledger_entries_for_note(deck_name, source_note_id)`
`DOMINATED` `selected = False`
`recommended_cards = 0`
    - `'bad'` `'unreviewed'`
`COGNITIVE_DEBT` `should_replace = False`

  - `GenerateCardsUseCase._apply_mutual_negative_boundaries`

  - `'optimal'`
-

  - `backend/app/services/inspect_material_service.py`
  - `retrieve_media_base64`
`CoreTheme.associated_diagrams` `diagram_roles`
  - `target_learner_level` `beginner`
`LearningPlanContract`
-
  - `backend/app/services/socratic_red_teaming_service.py` `backend/app/api/routes.py`
  - `POST
/anki/decks/{deck_name}/nodes/{concept_id}/socratic-challenge` `POST
/anki/decks/{deck_name}/nodes/{concept_id}/socratic-evaluate`
  - `>= 0.75` `MASTERED`
  - `OrphanGap` `SANITIZED`
  - `SUFFICIENT`
`⚠️ Trampa frecuente`

1.

`pytest backend/tests/test_deck_intelligence_service.py
backend/tests/test_inspection_and_prerequisites.py
backend/tests/test_thematic_coverage_audit.py backend/audits/test_l2_cognitive_linters.py
backend/audits/test_l4_llm_judge_evals.py backend/audits/test_l5_socratic_red_teaming.py -v`
2.

`pytest backend/audits/test_api_fuzzing_schemathesis.py -v`

-
  - `handleApproveInspection` `matched_note_id`
`obsolete_note_ids`
  - `pending_ledger_collisions`
  -
`1..8` `CoreTheme`
-
  - `api.getSanitationQueue`
  - `api.replaceNoteTransactionally`
`selectedAnkiCard` `inspection.source_note_id`
`api.exportToAnki`
  - `card.source_images`
`.shift()`
  - `SocraticNodeMasteryModal.tsx`
-

- `DeckStudioModal.tsx`
  `App.tsx`
- `AnkiCardPickerModal.tsx`
  `App.tsx`

1.
   `cd frontend && npx tsc -p tsconfig.app.json --noEmit --noUnusedLocals
--noUnusedParameters`
2. `oxlint`
   `cd frontend && npm run lint`
3.
   `cd frontend && npm run build`
4.
   `cd frontend && npm run test:e2e`

`PASS` `FAIL`

`test_deck_knowledge_store.py` `[
]` `sanitation_coverage_ledger` `stage_id`
`test_sanitation_service.py` `test_l1` `test_l3` `[ ]`
`depth_level ASC`
`test_deck_intelligence` `test_l2/l4/l5`
`schemathesis` `[ ]`

`tsc` `npm run lint` `npm run build` `npm run test:e2e`
`[ ]`